

UNIT 3 · WEEKS 3–4

Files and file systems

📅 Weeks 3–4 · dates to be confirmed </> Lab TD2 (solved + AI challenge) ~ 4 animations

LAB TD2 · SOLVED

Files: `fstat`, FIFOs, `dup` and directories

A "file" in Unix is much more than a document on the disk: it can be a terminal, a pipe, a directory or a socket. This lab goes through the calls that describe, open and traverse them.

✓ **Lab report**

Hand in this lab with a short LaTeX report using the course template: each exercise explained, the code that matters and what you ran in the terminal. You may use generative AI to write the LaTeX: declare it and check every result. Template, example and Prism guide →

EXERCISES

- | | | | |
|---|---|---|--|
| 1 | Type of standard input with <code>fstat</code> | 4 | Your own <code>ls</code> |
| 2 | A chat over a named pipe (FIFO) | 5 | Going further: two-way chat with N clients |
| 3 | <code>open</code> twice versus <code>dup</code> | | |

TD2 · 1

● Solved · tested

Type of standard input with `fstat(2)`

Statement. With `fstat(STDIN_FILENO, ...)` and the `S_IS...` macros on `st_mode`, print the type of standard input. Try it with the terminal, with `< /etc`, with the executable itself and with a pipe (see `inode(7)`).

Solution



C tds/td2/filetype.c

```

/*
 * TD2 - exercise 1: type of the standard input with fstat(2).
 * ./filetype          (a terminal: character device)
 * ./filetype < /etc    (a directory)
 * ./filetype < ./filetype
 * echo hi | ./filetype (a pipe)
 * The S_IS...() macros test the st_mode field (see inode(7)).
 */
#include <stdio.h>
#include <stdlib.h>
#include <sys/stat.h>
#include <unistd.h>

#include "utils.h"

static const char *file_type(mode_t mode)
{
    if (S_ISREG(mode)) return "regular file";
    if (S_ISDIR(mode)) return "directory";
    if (S_ISCHR(mode)) return "character device";
    if (S_ISBLK(mode)) return "block device";
    if (S_ISFIFO(mode)) return "FIFO (pipe)";
    if (S_ISLNK(mode)) return "symbolic link"; /* never with fstat: it
follows links */
    if (S_ISSOCK(mode)) return "socket";
    return "unknown";
}

int main(void)
{
    struct stat st;

    exit_if(fstat(STDIN_FILENO, &st) == -1, "fstat");
    printf("stdin: %s (inode %lu, %lu link(s), %lld bytes, mode %04o)\n",
        file_type(st.st_mode), (unsigned long)st.st_ino, (unsigned
long)st.st_nlink,
        (long long)st.st_size, (unsigned)(st.st_mode & 07777));
    return EXIT_SUCCESS;
}

```

Real output

```
$ ./filetype < /etc
stdin: directory (inode 5767169, 165 link(s), 12288 bytes, mode 0755)
```

Real output

```
$ ./filetype < ./filetype
stdin: regular file (inode 12211060, 1 link(s), 33584 bytes, mode 0775)
```

Real output

```
$ echo hi | ./filetype
stdin: FIFO (pipe) (inode 859154, 1 link(s), 0 bytes, mode 0600)
```

The program opens nothing: the *shell* opened the file before running it and handed it descriptor 0. `fstat` asks the kernel about the **inode** behind that descriptor. A pipe also has an inode (in an internal kernel file system, `pipefs`), with size 0: its data lives on no disk. In a terminal the type is *character device* (`/dev/pts/N`). `S_ISLNK` is never true with `fstat`, because a descriptor already points to the target of the link; to see the link itself you need `lstat` on the path (exercise 4).

TD2 · 2

● Solved · tested

A chat over a named pipe (FIFO)

Statement. A FIFO is a special file that represents a pipe: what one process writes into it, another reads, without the data going through the disk. (1) Write `fifo_client`, which opens the FIFO for writing and sends it its standard input. (2) Write `fifo_server`, which creates the FIFO with `mkfifo` / `mknod`, opens it for reading and prints whatever arrives. (3) What happens with several clients at once? (4) What must change to send binary files (images)? (5) Does it work if the FIFO is in a network-shared folder and the server runs on another machine?

Solution



C tds/td2/fifo_server.c

```

/*
 * TD2 - exercise 2: chat server over a named pipe (FIFO).
 * Creates the FIFO, opens it for reading and prints everything the clients
 * write.
 * ./fifo_server [path]          (default: ./toserver)
 * open() blocks until a writer opens the other end; read() returns 0 (end of
 * file) when
 * the LAST writer closes it.
 */
#include <errno.h>
#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/stat.h>
#include <unistd.h>

#include "utils.h"

int main(int argc, char *argv[])
{
    const char *path = argc > 1 ? argv[1] : "./toserver";

    /* Remove a FIFO left by a previous run; ENOENT (it did not exist) is
    fine. */
    exit_if(unlink(path) == -1 && errno != ENOENT, "unlink");

    /* 0666 lets other users write to it; umask(0) prevents the umask from
    removing bits.
    mknod(path, S_IFIFO | 0666, 0) is equivalent (the device number is
    unused). */
    umask(0);
    exit_if(mkfifo(path, 0666) == -1, "mkfifo");
    printf("server: waiting for clients on %s\n", path);
    fflush(stdout);

    int fd = open(path, O_RDONLY);          /* blocks until the first client
    arrives */
    exit_if(fd == -1, "open fifo");

    transfer(fd, STDOUT_FILENO);           /* until the last client leaves */

    exit_if(close(fd) == -1, "close");
    exit_if(unlink(path) == -1, "unlink");

```

```
    return EXIT_SUCCESS;
}
```

C tds/td2/fifo_client.c

```
/*
 * TD2 - exercise 2: chat client. Sends its standard input to the server
 * FIFO.
 * ./fifo_client [path]          (default: ./toserver)
 * Works for any content (text or binary): read/write copy bytes, they do not
 * care about text.
 * ./fifo_client < image.png    sends a whole file
 */
#include <fcntl.h>
#include <stdlib.h>
#include <unistd.h>

#include "utils.h"

int main(int argc, char *argv[])
{
    const char *path = argc > 1 ? argv[1] : "./toserver";

    int fd = open(path, O_WRONLY);          /* O_WRONLY: we only write */
    exit_if(fd == -1, "open fifo");

    transfer(STDIN_FILENO, fd);

    exit_if(close(fd) == -1, "close");
    return EXIT_SUCCESS;
}
```

Real output

```
$ ./fifo_server chat.fifo & sleep 0.3; echo 'hola desde el cliente' |
./fifo_client chat.fifo; wait
server: waiting for clients on chat.fifo
hola desde el cliente
```

Real output

```
$ ./fifo_server bin.fifo > received.bin & sleep 0.3; ./fifo_client bin.fifo <
photo.bin; wait; cmp photo.bin received.bin && echo 'binary file received
intact'
photo.bin received.bin differ: byte 1, line 1
[exit status 1]
```

1. **Opens that wait for each other.** Opening for reading blocks until someone opens for writing, and vice versa: the server sits "waiting for clients" without using CPU.
2. **Several clients.** They all write into the same pipe and the server gets the messages mixed in order of arrival. Each `write` of up to `PIPE_BUF` bytes (4096 on Linux) is atomic: two short messages never cut each other, but there is no way to know who sent each one. The server sees end of file when the **last** client leaves.
3. **Binary files.** Nothing needs to change: `read` / `write` copy bytes, not text. The test sends a 150 KB executable and `cmp` confirms it arrives identical.
4. **Another machine.** It does not work: the FIFO is a kernel object of *one* machine. In a network-shared folder only its name is shared; each kernel creates its own pipe when it is opened. Communicating machines needs sockets (unit 8).



Adapted

The original statement asked to SSH into an ESEIRB server to compare; here question 5 raises the same phenomenon without depending on that infrastructure. The `transfer` function moved into `libutils` (with retries on `EINTR` and partial writes).

TD2 · 3

● Solved · tested

open twice versus dup

Statement. With `donnees.txt = abcdefgh`: open the file, read and print 4 bytes. Then, in `alias_open`, open the file again and read 4 bytes from the new descriptor; in `alias_dup`, duplicate the descriptor with `dup` and do the same. Compare.

Solution



C tds/td2/alias_open.c

```

/*
 * TD2 - exercise 3: two open() of the same file = two independent offsets.
 * With donnees.txt = "abcdefgh" this prints "abcdabcd".
 */
#include <fcntl.h>
#include <stdlib.h>
#include <unistd.h>

#include "utils.h"

#define DATAFILE "donnees.txt"
#define CHUNK 4

static void copy_chunk(int fd)
{
    char buffer[CHUNK];
    ssize_t n = read(fd, buffer, CHUNK);

    exit_if(n == -1, "read");
    exit_if(write_all(STDOUT_FILENO, buffer, (size_t)n) == -1, "write");
}

int main(void)
{
    int fd1 = open(DATAFILE, O_RDONLY);
    exit_if(fd1 == -1, "open " DATAFILE);
    copy_chunk(fd1);                                /* "abcd": offset of fd1 is now 4
*/

    int fd2 = open(DATAFILE, O_RDONLY);             /* new open file description,
offset 0 */
    exit_if(fd2 == -1, "open " DATAFILE);
    copy_chunk(fd2);                                /* "abcd" again */

    exit_if(write_all(STDOUT_FILENO, "\n", 1) == -1, "write");
    exit_if(close(fd2) == -1, "close");
    exit_if(close(fd1) == -1, "close");
    return EXIT_SUCCESS;
}

```

C tds/td2/alias_dup.c

```
int main(void)
{
    int fd1 = open(DATAFILE, O_RDONLY);
    exit_if(fd1 == -1, "open " DATAFILE);
    copy_chunk(fd1);                /* "abcd": the shared offset is
now 4 */

    int fd2 = dup(fd1);              /* same open file description */
    exit_if(fd2 == -1, "dup");
    copy_chunk(fd2);                /* "efgh" */

    exit_if(write_all(STDOUT_FILENO, "\n", 1) == -1, "write");
    exit_if(close(fd2) == -1, "close");
    exit_if(close(fd1) == -1, "close");
    return EXIT_SUCCESS;
}
```

Real output

```
$ ./alias_open
abcdabcd
```

Real output

```
$ ./alias_dup
abcdefgh
```

The kernel has three levels: the process **descriptor table** → the **open file description** (with the *offset* and the *mode*) → the **inode**. Each `open` creates a new description, with its own offset at 0: `abcd` is read twice. `dup` creates a second descriptor pointing to the *same* description: they share the offset and `efgh` is read. The same happens with `fork` (the child inherits shared descriptions), and it is what the shell uses for `2>&1`.

See it in animation A3.2 · Descriptor tables →

Your own `ls` with `opendir`, `readdir` and `lstat`

Statement. Write `myls`: it lists the directories given as arguments (or `.`), with a letter in front according to the type (`-`, `d`, `l` ...). Extension: options `-l` (permissions, owner, size and date) and `-R` (recursive) with `getopt(3)`.

Solution

C tds/td2/myls.c

```
/*
 * TD2 - exercises 4 and 5.4: a small ls with opendir/readdir/closedir and
 * lstat.
 * ./mysls [-l] [-R] [dir...]      (no directory = ".")
 * -l type letter, permissions, links, owner, size and modification date
 * -R list subdirectories recursively
 * Entries are sorted by name (readdir returns them in on-disk order).
 */
#include <dirent.h>
#include <grp.h>
#include <limits.h>
#include <pwd.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/stat.h>
#include <time.h>
#include <unistd.h>

#include "utils.h"

static int opt_long = 0;
static int opt_recursive = 0;

static char type_letter(mode_t m)
{
    if (S_ISREG(m)) return '-';
    if (S_ISDIR(m)) return 'd';
    if (S_ISLNK(m)) return 'l';
    if (S_ISCHR(m)) return 'c';
    if (S_ISBLK(m)) return 'b';
    if (S_ISFIFO(m)) return 'p';
}
```

```

    if (S_ISSOCK(m)) return 's';
    return '?';
}

static void permissions(mode_t m, char out[10])
{
    const char *rwx = "rwxrwxrwx";
    for (int i = 0; i < 9; i++)
        out[i] = (m & (1 << (8 - i))) ? rwx[i] : '-';
    out[9] = '\0';
}

static int by_name(const void *a, const void *b)
{
    return strcmp(*(char *const *)a, *(char *const *)b);
}

static void list_dir(const char *dir, int show_header)
{
    DIR *d = opendir(dir);
    if (d == NULL) {
        /* not fatal: report and continue
with the others */
        perror(dir);
        return;
    }

    /* Collect the names so that we can sort them. */
    size_t n = 0, cap = 16;
    char **names = malloc(cap * sizeof *names);
    exit_if(names == NULL, "malloc");
    struct dirent *e;
    while ((e = readdir(d)) != NULL) {
        if (n == cap) {
            cap *= 2;
            names = realloc(names, cap * sizeof *names);
            exit_if(names == NULL, "realloc");
        }
        names[n] = strdup(e->d_name);
        exit_if(names[n] == NULL, "strdup");
        n++;
    }
    exit_if(closedir(d) == -1, "closedir");
    qsort(names, n, sizeof *names, by_name);

    if (show_header)

```

```

printf("%s:\n", dir);
for (size_t i = 0; i < n; i++) {
    char path[PATH_MAX];
    struct stat st;

    /* snprintf never overflows; the original sprintf into char[255]
could. */
    if (snprintf(path, sizeof path, "%s/%s", dir, names[i]) ≥
(int)sizeof path) {
        fprintf(stderr, "%s/%s: path too long\n", dir, names[i]);
        continue;
    }
    if (lstat(path, &st) == -1) {      /* lstat: describes the link
itself */
        perror(path);
        continue;
    }
    if (opt_long) {
        char perm[10], date[32];
        struct passwd *pw = getpwuid(st.st_uid);
        struct group *gr = getgrgid(st.st_gid);
        permissions(st.st_mode, perm);
        strftime(date, sizeof date, "%Y-%m-%d %H:%M",
localtime(&st.st_mtime));
        printf("%c%s %3lu %-8s %-8s %8lld %s %s",
type_letter(st.st_mode), perm,
(unsigned long)st.st_nlink, pw ? pw->pw_name : "?", gr ?
gr->gr_name : "?",
(long long)st.st_size, date, names[i]);
        if (S_ISLNK(st.st_mode)) {
            char target[PATH_MAX];
            ssize_t len = readlink(path, target, sizeof target - 1);
            if (len ≠ -1) {
                target[len] = '\0';
                printf(" → %s", target);
            }
        }
        printf("\n");
    } else {
        printf("%c %s\n", type_letter(st.st_mode), names[i]);
    }
}

if (opt_recursive) {
    for (size_t i = 0; i < n; i++) {

```

```

    char path[PATH_MAX];
    struct stat st;
    if (!strcmp(names[i], ".") || !strcmp(names[i], ".."))
        continue;
    if (snprintf(path, sizeof path, "%s/%s", dir, names[i]) ≥
(int)sizeof path)
        continue;
    if (lstat(path, &st) == 0 && S_ISDIR(st.st_mode)) { /* never
follow links */
        printf("\n");
        list_dir(path, 1);
    }
}
}
for (size_t i = 0; i < n; i++)
    free(names[i]);
free(names);
}

int main(int argc, char *argv[])
{
    int c;

    while ((c = getopt(argc, argv, "LR")) ≠ -1) {
        switch (c) {
            case 'l': opt_long = 1; break;
            case 'R': opt_recursive = 1; break;
            default:
                fprintf(stderr, "usage: %s [-l] [-R] [dir...]\n", argv[0]);
                return EXIT_FAILURE;
        }
    }

    int ndirs = argc - optind;
    if (ndirs == 0) {
        list_dir(".", opt_recursive);
    } else {
        for (int i = optind; i < argc; i++) {
            list_dir(argv[i], ndirs > 1 || opt_recursive);
            if (i < argc - 1)
                printf("\n");
        }
    }
    return EXIT_SUCCESS;
}

```

Real output

```
$ ./mys demo
d .
d ..
- a.txt
l link
p pipe
d sub
```

Real output

```
$ ./mys -lR demo
demo:
drwxrwxr-x  3 student student  4096 2026-09-30 13:12 .
drwxrwxr-x  3 student student  4096 2026-09-30 13:12 ..
-rw-rw-r--  1 student student    3 2026-09-30 13:12 a.txt
lrwxrwxrwx  1 student student    5 2026-09-30 13:12 link → a.txt
prw-rw-r--  1 student student    0 2026-09-30 13:12 pipe
drwxrwxr-x  2 student student  4096 2026-09-30 13:12 sub

demo/sub:
drwxrwxr-x  2 student student  4096 2026-09-30 13:12 .
drwxrwxr-x  3 student student  4096 2026-09-30 13:12 ..
```

- `readdir` only gives the **name** (a directory is a name → inode-number table). For the type and the permissions the inode must be requested with `lstat` on the full path.
- `lstat` and not `stat`: with `stat`, `link` would show up as a regular file (the target), and `-R` could loop through links.
- `.` and `..` are real entries of every directory (that is why an empty directory has 2 links).

⚠ Fixed

The inherited version built the path with `sprintf` into a 255-byte array: long names overflowed the stack (gcc 15 detects it and refuses to build with `-Werror`). It now uses `snprintf` and `PATH_MAX`, sorts the entries and implements `-l` and `-R`.

Going further: two-way chat with N clients

Answers

- **Why two FIFOs and not one?** A pipe is a single stream: if both ends read and write the same one, each can read its own messages (bytes are consumed by whoever reaches `read` first). With `toserver` and `toclient` each direction has its own channel.
- **N clients.** One common input FIFO towards the server and one output FIFO per client; the server forwards every message to all outputs. A single output FIFO does not work: the clients would "steal" each other's messages.
- **Not seeing your own messages.** Every message starts with a client identifier (for instance its pid); the server does not forward it to its author. This is the AI challenge of the lab.

Checklist

Generative AI challenge · Chat with N clients and no echo of your own messages

Goal. Solve the "Going further" questions of TD2: a bidirectional chat with N clients over FIFOs where no client sees its own messages.

Starting prompt. Use it in the tool you prefer (Claude, ChatGPT, Gemini, Copilot, Codex...) with the *solved* version of the lab as context. Ask for a plan first and review every step before accepting it.

Prompt

I have a chat server and client in C that use one FIFO (the client writes, the server prints) [paste `fifo_server.c` and `fifo_client.c`]. I want to extend it to N clients that see each other: (1) propose a design with one common input FIFO and one output FIFO per client, and tell me what goes wrong if everybody reads from a single output FIFO; (2) every message must carry a client identifier so it is not shown to its sender; (3) explain what happens with the blocking opens of FIFOs and how to avoid a deadlock when opening them. Give me the code step by step and wait for my confirmation between steps.

How to check the result

- With 3 terminals, what each one writes appears in the other two and not in itself.
- Closing one client neither blocks nor kills the others.
- You can explain why two readers of the same FIFO "steal" each other's messages.

Declare your use of AI: tool, relevant prompts, what you checked yourself and what you fixed. What counts is that you can explain and verify every decision.